하흥주

프론트엔드 개발자

포트폴리오

기술 스택

client
- React with Vite, Bun
- TanStack Query, Router, Table, Virtual
- Zod
- Zustand

api
- Express.js
- Prisma, PostgreSQL

deployment
- Railway
- S3, CloudFront

documentation
- Typst

주요 프로젝트

Arbor The Tree - 진도 및 오답 관리 시스템

IUA React UI - 디자인 라이브러리

Terminal To Blog - 자동 업데이트 블로그

Arbor The Tree - 진도 및 오답 관리 시스템

기획 의도

강사 재직 시절, 구글 시트와 파이썬을 이용해 사용한 진도 및 오답 관리 시스템을 웹앱으로 제작

주요 기능

그리드 인풋 및 다중 입력

특수 문자 입력으로 자동 입력, 자동 삭제를 하여 이용 편의성 증진

가상 스크롤로 그리드 인풋의 반응 속도를 4초에서 0.3초로 단축하며, 유효성 검사는 유지

오답 그리드 및 다중 선택

가상 스크롤로 그리드 체크박스의 반응 속도를 0.3초에서 0.1초로 단축

오답 체크를 할 때마다 진도 캐시를 무효화하여 해당 문제 묶음의 완료 여부를 갱신

주요 기능 외 해결한 문제

서버 응답 속도 개선: 7s -> 0.3s

typst compiler를 활용한 pdf 생성으로, react-pdf 대비 생성시간 90% 단축

props drilling, re-render 문제를 컴포넌트별 Zustand 스토어로 해결

Arbor The Tree - 주요 기능

1. 그리드 인풋 및 다중 입력

그리드 인풋

- 구글 스프레드 시트와 유사한 UI
- custom data attribute `data-coordinate`을 이용한 키보드 내비게이션

다중 입력

- 셀에 / 입력 시, 자동으로 알맞는 값을 오버레이, 그 밑의 빈 셀 모두에 같은 값 오버레이
- 문제 번호에 ~ 입력 시, 해당 번호들을 자동 완성 (예: `확인 1~10`)
- 문제 번호 끝에 - 입력 시, 해당 셀부터 끝까지의 문제 번호 삭제 (예: `122-`)

유효성 검사

- 문제 번호 없이 다른 열의 값 기입 시 오류 알림
- 자동 완성할 것이 더 이상 없을 때 /를 사용 시 오류 알림
- 문제 번호가 없는 행에서는 자동 완성 비활성화

해결한 문제

- 빠른 반응을 위해 유효성 검사 셀 수를 최소화하려 하였으나, 해당 로직이 너무 복잡해지고 버그가 많아짐
- 모든 입력마다 전체 셀 유효성 검사를 하되, 가상 스크롤(TanStack Virtual)을 이용해 돔 노드를 최소화하여 20.38ms에서 2.28ms로 반응 시간 89% 단축(`console.time`으로 측정)

Figure 1: Grid Input

문제집 등록 페이지
테스트 계정 이용 방법

Arbor The Tree - 주요 기능

2. 서버 사이드 pdf 생성

최초 계획: 클라이언트 사이드 pdf 생성

- `react-pdf/renderer`를 이용한 클라이언트 사이드 pdf 생성 계획
- 해당 작업이 브라우저 메인 스레드를 차지함
- 때문에 30매 이상의 pdf 생성 시, 웹 워커 사용이 권장됨
- 의문: pdf는 브라우저와 상관 없는데 웹 워커까지 써야 하나?

개선안: compile typst to pdf

- request > create typst file > compile to pdf > respond

해결한 문제

- 문제 1: typst는 npm install로 설치 되지 않음
- 문제 2: typst, font 파일은 npm build로 빌드되지 않음
- 해결: `railway.json`에 buildCommand bash script를 작성
  1. typst compiler 설치
  2. `npm run build`
  3. build에서 제외되는 파일들 dist 폴더로 이동
     - typst template files
     - font

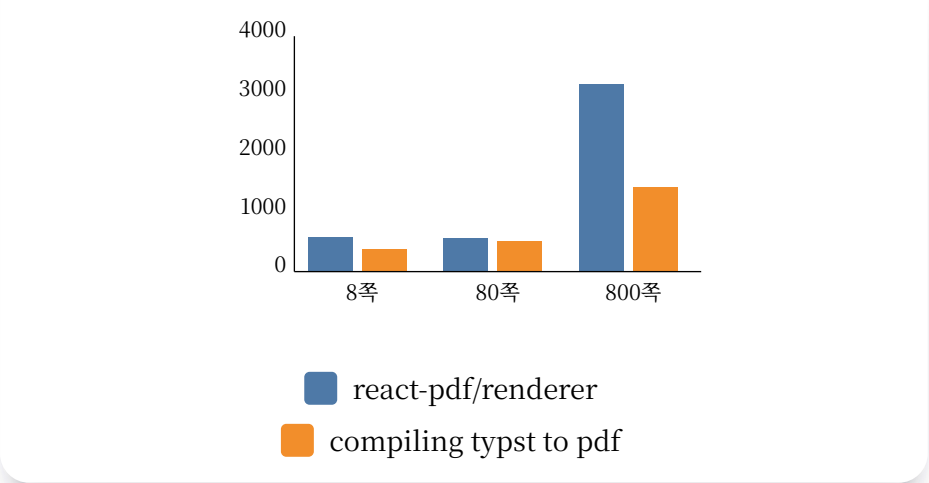

Figure 2: Performance Comparison

성능 비교 테스트 페이지

Arbor The Tree - 주요 기능 외 해결한 문제

1. 컴포넌트별 Zustand 스토어

최초 계획

- 복잡한 공통 컴포넌트 제작 시 props-drilling을 막기 위해 Context API를 사용
- 상태를 하나 추가할 때마다 4곳의 코드 수정해야 함
  - 1. 컴포넌트 interface에 해당 타입 추가
  - 2. 컴포넌트 props에 해당 매개변수 추가
  - 3. context provider의 value에 해당 상태 추가 주입
  - 4. context type 수정
- 어느 상태가 변해도 context 구독 중인 모든 컴포넌트가 리렌더 됨

개선안

- 컴포넌트 별 스토어 제작
- 상태를 하나 추가할 때마다 2곳의 코드만 수정하면 됨
  - 1. 컴포넌트 interface에 해당 타입 추가
  - 2. 컴포넌트 props에 해당 매개변수 추가
  - props 전체가 store provider에 넘겨지기에 추가로 수정할 부분 없음
- 자식 컴포넌트가 사용하는 상태가 변경될 때만 리렌더가 됨

해결한 문제

- 문제: 같은 버튼 컴포넌트여도 스토어를 공유해선 안 됨
- 해결: 스토어 생성을 컴포넌트 내에서 하고 이를 useState에 저장
  1. TODO: 구체적 단계 적어야
  1. TODO: 구체적 단계 적어야
  1. TODO: 구체적 단계 적어야
  1. TODO: 구체적 단계 적어야
  1. TODO: 구체적 단계 적어야

관련 내용 도식화

Arbor The Tree - 주요 기능 외 해결한 문제

2. 서버 응답 속도 개선

최초 계획

- api server on Railway, db server on Neon
- 문제: TODO 채워 넣어야
- 문제: TODO 채워 넣어야
- 문제: TODO 채워 넣어야
- 문제: TODO 채워 넣어야
- 문제: TODO 채워 넣어야

개선안

- 컴포넌트 별 스토어 제작
- 상태를 하나 추가할 때마다 2곳의 코드만 수정하면 됨
  - 1. 컴포넌트 interface에 해당 타입 추가
  - 2. 컴포넌트 props에 해당 매개변수 추가
  - props 전체가 store provider에 넘겨지기에 추가로 수정할 부분 없음
- 자식 컴포넌트가 사용하는 상태가 변경될 때만 리렌더가 됨

관련 스크린샷

Arbor The Tree - 이용 방법

링크

- https://arbor-the-tree-production.up.railway.app/
- https://github.com/ThePott/arbor-the-tree
- 테스트 계정 로그인 페이지
- pdf 생성 성능 비교 페이지

테스트 계정

계정	ID	Password
원장	`test12@test.test`	`test1234!@#$`
실장	`test13@test.test`	`test1234!@#$`
학생	`test14@test.test`	`test1234!@#$`

IUA React UI - 디자인 라이브러리

기획 의도

개발 초기 단계에서 공통 컴포넌트를 개발하는 시간을 단축하기 위해, 이전 프로젝트의 공통 컴포넌트를 npm 패키지로 배포

컬러 테마 뿐 아니라 세부 로직도 사용자가 자유롭게 바꿀 수 있게 제작

주요 기능

커스터마이징 가능한 컬러 스킴

사용자가 세세하게 변경할 수 있도록 node_modules에가 아닌 utils 폴더에 해당 컴포넌트 파일이 노출된 채 복사되도록 npx 명령어 제작

컴포넌트별 페이지를 만들어 각 속성에 따른 생김새 변화를 확인

주요 기능 외 해결한 문제

TanStack Router CLI 가 아무것도 하지 않고 바로 종료되는 버그 발견, 수정 후 PR, 현재 병합 대기 중

Terminal To Blog - 자동 업데이트 블로그

기획 의도

모든 문서 작업을 터미널에서 하되, 터미널에 익숙하지 않은 사람도 쉽게 열람할 수 있는 블로그 제작

source of truth를 깃헙 레포로 하여 블로그의 내용을 자동 업데이트

IUA React UI의 실사용

주요 기능

동적 라우팅을 통해 레포 내에서의 새로운 폴더 생성을 자동으로 반영하여 새 라우트 제작